

State-of-the-Art AI Risk Management for E-Commerce: Cost-Sensitive Optimization, Graph-Sequential Architectures, and Defensive Evaluation Constraints

Introduction to the Modern Risk Management Paradigm

In the contemporary landscape of digital commerce and financial services, the proliferation of sophisticated fraud vectors presents an existential threat to the operational margins of payment gateways, merchants, and financial institutions. As the industry transitions from static, rule-based heuristics to machine learning-driven defense mechanisms, a critical paradigm shift has emerged regarding how these systems are architected, evaluated, and deployed. The mandate for modern Artificial Intelligence (AI) risk managers—particularly within high-stakes environments such as competitive hackathons sponsored by leading payment processors like Razorpay—is no longer simply to classify transactions as legitimate or fraudulent. Instead, the objective is to execute these classifications with a rigorous, financially grounded understanding of the asymmetric costs of misclassification, while maintaining a strictly defensive posture. The core challenge in deploying state-of-the-art fraud detectors, returns-abuse verifiers, and automated response systems lies entirely within the discipline of evaluation. Traditional machine learning metrics, such as overarching accuracy or static False Positive Rates (FPR), are fundamentally inadequate when applied to highly imbalanced datasets where the minority class (fraud) carries a disproportionate financial penalty compared to the majority class (legitimate transactions)¹. However, the cost of a false positive—incorrectly declining a loyal, high-lifetime-value customer—extends far beyond the lost interchange fee or the immediate product margin; it inflicts long-term damage on customer retention, drives negative brand equity, and mathematically degrades the lifetime value (LTV) of the consumer base³. Consequently, deploying a robust AI risk manager requires the seamless integration of cost-sensitive learning, dynamic decision threshold optimization, and Bayes Minimum Risk (BMR) frameworks to present honest metrics that explicitly quantify the false-positive cost⁶. Furthermore, in an environment where adversarial threats continuously evolve, the deployment of defensive AI must remain strictly isolated from offense-capable generation. A rigorous defense-only posture relies on Differential Privacy (DP), anomaly detection, and graph-based entity resolution to identify malicious network topologies without training models that could be reverse-engineered to bypass security controls or generate synthetic fraud identities⁷. This report provides an exhaustive, expert-level blueprint for architecting a state-of-the-art AI Risk

Manager. It encompasses the mathematical foundations of cost-sensitive evaluation, the structural economics of e-commerce fraud and Return-to-Origin (RTO) abuse, advanced heterogeneous graph neural network (GNN) architectures, and the implementation of privacy-preserving machine learning to guarantee a strictly defensive operational mode.

The Economics of Fraud Detection and Evaluation

Discipline

The fundamental flaw in traditional fraud detection evaluation is the implicit assumption that all classification errors carry equal weight. In reality, the financial impact of a False Negative (a missed fraudulent transaction resulting in a chargeback, network penalty, and lost inventory) is vastly different from that of a False Positive (a blocked legitimate transaction resulting in lost margin and customer friction)¹. To clear the highest evaluation bars in risk management, the AI system must abandon accuracy as a primary metric and instead adopt a profit-driven, cost-sensitive learning framework that reports precision, recall, and the explicit financial cost of classification errors.

The Inadequacy of Traditional Machine Learning Metrics

In highly imbalanced datasets, such as credit card fraud or e-commerce returns abuse, fraudulent instances often comprise less than one percent of total transaction volume². Under these conditions, a naive classification model that simply predicts every transaction as legitimate will effortlessly achieve over 99 percent accuracy. This renders standard accuracy entirely useless as an evaluation metric, as it obscures catastrophic financial losses beneath a veneer of high statistical performance¹. While metrics like Precision, Recall, and the F1-score provide a vastly superior understanding of the model's performance on the minority class, they still fail to capture the bottom-line financial reality required by enterprise risk managers. Precision dictates the proportion of flagged orders that are actually fraudulent, ensuring that the system is not overwhelmed by false alarms, while Recall measures the proportion of total fraud successfully intercepted by the system⁴. The F1-score harmonic mean balances these two metrics, providing a unified benchmark⁴. However, for decades, the False Positive Ratio (FPR) was the predominant industry standard for evaluating rule-based decision engines³. The FPR represents the ratio of legitimate transactions caught by the rule divided by the actual

frauds detected, expressed mathematically as
$$FPR = \frac{FP}{FP+TN} \quad 3.$$

Despite its widespread use, FPR is merely a comparative metric of model success and failure; it does not reflect the actual monetary value of the transactions being evaluated. A model might achieve a theoretically excellent low FPR by effectively blocking high volumes of zero-dollar "tester" authorizations utilized by botnets, but simultaneously block a small number of high-value legitimate purchases. In this scenario, the model would produce a negative Return on Investment (ROI) due to the substantial loss of interchange fees and product margins on the blocked legitimate orders, rendering the low FPR mathematically misleading and financially ruinous³.

Evaluation Metric	Traditional Optimization Focus	Cost-Sensitive Business Implication	Failure Mode in Imbalanced Fraud Data
Accuracy	Maximize total correct predictions across all classes.	Ignored entirely by risk professionals due to severe class imbalance bias.	A model predicting 100% "Legitimate" achieves 99% accuracy but allows all fraud to pass, maximizing financial loss.
False Positive Rate (FPR)	Keep below a static threshold (e.g., 1%).	Must be weighed against the financial severity of the blocked transaction.	High FPR on low-value items may be acceptable; high FPR on high-margin items is catastrophic. Hides monetary impact.
Precision	Ensure high confidence in flags to minimize manual review.	High precision means few good customers are insulted, minimizing False Positive cost.	Over-optimizing for precision often destroys recall, allowing sophisticated, high-cost fraud vectors to succeed undetected.
Recall	Catch as much anomalous activity as possible.	High recall minimizes False Negative costs (chargebacks, lost COGS).	Over-optimizing for recall destroys precision, resulting in mass customer friction, lost sales, and unmanageable manual review queues.
Decision	Static at 0.5 .	Dynamic (e.g.,	Assumes equal penalty for False

Threshold		0.05 to 0.15) based entirely on the underlying cost matrix.	Positives and False Negatives, leading to suboptimal financial outcomes.
-----------	--	---	---

Mathematical Formulation of Cost-Sensitive Learning

Cost-Sensitive Learning, originally formalized in the foundational theories of Elkan, operates on the premise that the risk cost due to misclassification differs fundamentally for each class, establishing optimal decision criteria based on Bayesian decision theory¹. The objective of the machine learning model is no longer to minimize the error rate, but to explicitly minimize the total expected financial loss across the portfolio.

For a binary classification problem, let the cost matrix C define the cost of predicting class i when the true class is j . The expected risk (or loss) of predicting class i for a given instance x is defined as the sum of the probabilities of each true class multiplied by the respective cost of misclassification:

$$L(x, i) = \sum_j P(j|x)C(i, j)$$

Where $P(j|x)$ is the posterior probability of the true class j given the input features x ¹³.

The optimal prediction for the instance x is the class i that mathematically minimizes this expected cost. In the specific context of fraud detection and chargeback management, the costs within the confusion matrix can be categorized and quantified as follows:

1. **True Negative** ($C(0, 0)$): The cost of approving a legitimate transaction. This is typically zero, or a negative cost representing the profit margin and interchange fee earned by the merchant and gateway.
2. **True Positive** ($C(1, 1)$): The cost of correctly blocking a fraudulent transaction. While there is a marginal computational and operational cost to executing the check, this represents a net savings by avoiding chargebacks and lost goods.
3. **False Negative** ($C(0, 1)$): The explicit cost of approving a fraudulent transaction. This encompasses a cascading series of losses: the Cost of Goods Sold (COGS), shipping and fulfillment expenses, the non-refundable chargeback administration fee imposed by the card network, and potential compliance penalties if fraud ratios exceed network

thresholds³.

4. **False Positive ($C(1, 0)$)**: The true cost of blocking a legitimate transaction. This represents the immediate lost product margin, the lost interchange fee, and the probabilistic loss of future customer lifetime value (LTV) due to churn induced by payment friction³.

Empirical research across the financial services sector indicates that the cost of a False Negative is traditionally estimated to be ten to one hundred times higher than the immediate, transactional cost of a False Positive¹. However, a truly honest evaluation framework—one that clears the rigorous bar of top-tier hackathons—must dynamically calculate the False Positive cost by factoring in lifetime value and secondary network effects. The actual ROI of any fraud decision rule, neural network, or automated response system can be quantified by balancing these outcomes into a unified financial equation:

$$ROI = CP + CNP_LA + CNP_HA - LI$$

Where CP represents Chargebacks Prevented (the administrative savings), CNP_LA represents Card Not Present Low Amount savings (blocking fraud where the value is less than the chargeback fee, thereby saving the whole purchase amount), CNP_HA represents Card Not Present High Amount savings (savings limited to the chargeback fee, assuming the merchant can recoup the purchase price via representment), and LI represents Lost Interchange, which serves as the quantitative penalty for the False Positive³.

Threshold Optimization and Bayes Minimum Risk (BMR)

Most out-of-the-box machine learning classifiers, including deep neural networks and gradient boosting ensembles like Extreme Gradient Boosting (XGBoost), output a continuous probability score between 0.0 and 1.0 , defaulting to a binary decision threshold of 0.5 ¹. This default threshold inherently assumes symmetric costs (i.e., $C(0, 1) = C(1, 0)$) and balanced class priors, an assumption that is mathematically invalid and financially dangerous in the context of fraud detection¹. Utilizing a 0.5 threshold will inevitably misclassify high-risk transactions that score, for example, 0.49 , classifying them as legitimate and resulting in highly costly False Negatives¹.

To clear the evaluation bar of reporting "honest metrics," the system must implement Bayes Minimum Risk (BMR) threshold optimization. BMR is an advanced decision model that quantifies the tradeoffs between various decisions by finding the exact probabilistic threshold that minimizes the expected financial loss over a validation dataset⁶.

If the cost of a False Negative is determined by the business to be ten times the cost of a False

Positive, the optimal decision threshold τ shifts significantly downward. Extensive studies on the highly imbalanced IEEE-CIS fraud detection dataset have demonstrated that adjusting the decision threshold from the default 0.5 to empirically derived optimal values (e.g., 0.08 or 0.0321) drastically reduces total operational costs while effectively balancing precision and recall via metrics like the Area Under the Precision-Recall Curve (AUPRC) or the Geometric Mean (G-Mean)¹. Under the BMR framework, the decision rule is fundamentally altered:

Predict Fraud if $P(Fraud|x) > \tau_{optimal}$

By decoupling the probability estimation of the machine learning model from the final classification decision, the AI Risk Manager can dynamically adjust to changing business requirements, fluctuating risk appetites, and seasonal cost variations (such as the heightened cost of false positives during peak holiday shopping seasons)¹⁰.

Cost Component	Variable	Financial Calculation	Hackathon Evaluation Impact
False Negative Cost	$C(0, 1)$	COGS + Shippin	Demonstrates understanding of total fraud exposure beyond just the transaction amount.
False Positive Cost	$C(1, 0)$	Lost Margin + (C	Proves "honest metrics" by penalizing the model for insulting good customers.
BMR Optimal Threshold	$\tau_{optimal}$	$\arg \min_{\tau} \sum [P(y =$	Shows advanced ML evaluation discipline by abandoning the default 0.5 threshold.
System ROI	ROI	Total Savings from TP - Total Losses	Provides a definitive, dollar-value metric

		from <i>FP</i>	for the model's success, superseding F1-scores.
--	--	----------------	---

E-Commerce Fraud Vectors and the Return to Origin (RTO) Crisis

While standard credit card chargeback fraud remains a pervasive threat in mature digital economies, emerging markets present unique structural challenges that require highly specialized AI Risk Managers. In the Indian e-commerce ecosystem, which is the primary operational theater for platforms like Razorpay, Cash-on-Delivery (COD) remains the overwhelmingly dominant payment method, accounting for approximately 60% to 65% of all retail orders¹⁶. This intense consumer preference for COD successfully mitigates the risk of traditional payment fraud and card testing, but it introduces a massive logistical and financial vulnerability known as Return to Origin (RTO).

The Anatomy and Cascading Costs of RTO Abuse

Return to Origin occurs when a dispatched order fails to reach the buyer, is refused upon delivery, or is cancelled in transit, and is subsequently returned to the merchant's warehouse¹⁸. Alarmingly, industry data reveals that 25% to 30% of COD orders end in an RTO state, compared to a mere 2% to 3% failure rate for prepaid transactions¹⁷. This staggering disparity underscores the lack of buyer commitment inherent in the COD model, combined with widespread address quality issues and first-party fraud.

RTO can be driven by innocent logistical failures, such as customer unavailability or courier inefficiencies, but it is heavily exploited through malicious intent. Fraudsters utilize synthetic identities, competitor sabotage tactics (draining a rival's logistics budget by ordering and refusing goods), and deliberate first-party fraud to exploit the system. The financial cascade of an RTO event is devastating to e-commerce unit economics, systematically eroding profit margins through three primary vectors:

- 1. Sunk Logistics Expenditures:** The merchant is forced to absorb the costs of both the forward delivery attempt and the reverse logistics pipeline, generating a net financial loss on a transaction that yielded zero revenue¹⁹.
- 2. Inventory Blocking and Opportunity Cost:** The physical product is tied up in the logistics network for weeks, rendering it unavailable for purchase by legitimate buyers. This is particularly damaging for high-turnover, fast-fashion, or seasonal inventory where product shelf-life is strictly limited⁵.
- 3. Physical Degradation and Operational Overhead:** Returned items require rigorous physical quality checks, extensive repackaging, and frequently suffer damage in transit, further eroding their resale value and driving up operational labor costs⁵.

Crucially, analytics reveal that over 90% of RTOs originate from behavioral signals present at

the initial checkout stage, rather than mid-mile logistics failures. These signals include intentionally malformed addresses, fake phone numbers, localized pin code anomalies, or impulsive COD selections⁵. Therefore, the AI Risk Manager must act not as a reactive logistics tool, but as an early-warning predictive engine, scoring the RTO risk of a transaction in real-time before the order is confirmed and routed to the warehouse for fulfillment.

Auto-Responders and the Implementation of Dynamic Friction

A core requirement for a comprehensive AI Risk Manager is the deployment of an "auto-responder" that takes decisive, defensive action when transaction risk crosses an optimized threshold. In the context of RTO and COD fraud, outright declining an order based on a probabilistic score carries an exceptionally high false-positive cost. Therefore, the most effective auto-responder strategy is the implementation of *Dynamic Friction*²¹.

Instead of a rigid binary "approve or decline" decision, the system utilizes the machine learning risk score to introduce targeted verification steps for high-risk profiles while maintaining a seamless, frictionless checkout experience for trusted users. If an order scores above the

calibrated high-risk threshold τ_{high} , the auto-responder can execute several tiered defensive actions:

- **COD Blocking and Payment Method Nudging:** Automatically disabling the Cash-on-Delivery option entirely for high-risk pin codes or anomalous user profiles, forcing the user to select a prepaid option if they wish to proceed. This shifts the risk away from the merchant¹⁸.
- **Part-Pay COD / Dynamic Fees:** Requiring the buyer to pay a small upfront fee (e.g., 5% to 10% of the total order value) via UPI or card to ensure they have "skin in the game." This psychological friction highly deters casual cancellations, impulse buying, and automated bot attacks while keeping the COD option partially alive¹⁷.
- **Automated Multi-Channel Verification:** Triggering immediate Interactive Voice Response (IVR) calls, AI voice agents, or WhatsApp API confirmations to validate the user's explicit intent and exact address details before the order is processed into the Order Management System (OMS)¹⁸.

By implementing dynamic friction, the AI Risk Manager explicitly and intelligently manages the false-positive cost. If a legitimate customer is mistakenly flagged as high-risk, they are not outright banned from the platform; rather, they are merely asked to verify their intent via a WhatsApp message or pay a nominal deposit. This preserves the potential sale, protects the lifetime value of the customer, and defends the merchant's logistics budget against RTO losses.

State-of-the-Art Architecture: Graph-Sequential Modeling

Traditional fraud detection systems have historically relied on manually curated rule engines (e.g., blocking all transactions over \$500 from a specific geographic region) or basic tabular machine learning models such as Logistic Regression or Random Forests. While gradient boosting frameworks like XGBoost remain highly effective baselines—especially when optimized with cost-sensitive learning parameters like `scale_pos_weight`—they inherently treat

each transaction as an isolated, independent event. This fundamental limitation prevents them from capturing the complex, collusive topologies and coordinated attacks utilized by modern, sophisticated fraud rings.

To achieve state-of-the-art performance and dominate competitive evaluations, the AI Risk Manager must leverage a deep learning architecture capable of modeling both multi-relational entity interactions and temporal behavioral sequences. A prime, production-tested example of this paradigm is the TRUST (Transaction Risk via Unified Sequence and Topology) framework, recently developed and deployed at massive scale in the Indian e-commerce market to process millions of transactions daily with sub-second latency¹⁶.

Heterogeneous Graph Neural Networks (HGNNs) for Topology

Mapping

Modern fraudsters continuously adapt their techniques by utilizing synthetic identities, rotating IP addresses, and employing advanced device emulators to evade tabular detection rules.

However, underlying network connections and shared resources inevitably remain. A Heterogeneous Graph Neural Network (HGNN) is uniquely suited to uncover these hidden, structural linkages by mapping the entire transaction ecosystem¹⁶.

Within this architecture, the e-commerce ecosystem is modeled as a massive heterogeneous

graph $G = (V, E)$. The node set V comprises distinct, multi-modal entity types, primarily users (V_U) and suppliers/merchants (V_S)¹⁶. Each node is initialized with a high-dimensional feature vector encompassing behavioral attributes (e.g., order frequency, past cancellation rates, delivery success history, region-specific signals, and address quality scores)¹⁶.

The edges (E) in the graph represent various relational interactions and identity overlaps:

- **User-User Edges:** Created when two seemingly distinct user accounts share a high-fidelity identity attribute, such as a physical device ID, Google Advertising ID (GAID), bank account number, or an exact geospatial coordinate¹⁶. These edges are critical for surfacing account duplication, botnet clustering, and organized fraud rings that tabular models completely miss.
- **Supplier-Supplier Edges:** Capture potential collusion between sellers. An edge is formed when merchants share tax registration IDs, overlapping bank accounts, or exhibit significant, anomalous overlap in their transacting user base (indicative of circular trading or triangulation fraud)¹⁶.
- **User-Supplier Edges:** Represent the transactional history, engagement metrics, and dispute incidences between buyers and sellers, mapping the direct flow of commerce and risk¹⁶.

By applying advanced message-passing algorithms across this heterogeneous graph, the HGNN aggregates feature representations from a node's extended topological neighborhood. This allows the model to effectively identify "guilt by association." Even if a newly registered user account presents a severe cold-start problem with zero transaction history, the GNN can confidently assign a high risk score if the account shares a device ID or sub-network linkage

with a known, previously banned fraud ring¹⁶.

Transformer-Based Sequence Encoders for Behavioral Temporal Analysis

While graph structures beautifully capture spatial topology and static connections, they can struggle to interpret the rapid nuances of temporal behavior. Fraudulent activity often follows highly specific chronological patterns, such as a long-dormant account suddenly exhibiting hyper-velocity purchasing behavior, or an account initiating multiple high-value transactions immediately following a password reset and shipping address change.

To capture this temporal dynamic, the HGNN embeddings are fused into a unified pipeline with a Transformer-based sequence encoder¹⁶. The sequence encoder processes the chronological, time-series log of a user's interactions on the platform (e.g., micro-interactions like clicks, cart additions, address edits, time spent on the checkout page, and typing cadence). By utilizing temporal positional encoding mechanisms, the Transformer isolates sequential behavioral anomalies that deviate significantly from legitimate shopping patterns¹⁶.

The unified GNN-Transformer cascade architecture provides a highly accurate, robust, and nuanced risk score. When either modality is sparse—for instance, a user with no graph connections but highly suspicious, bot-like temporal behavior, or a user with normal checkout behavior but highly suspicious graph linkages to known bad actors—the complementary modality compensates. This unified approach yields substantial absolute precision improvements over baseline gradient boosting models while operating well within the strict ~25 millisecond inference latency constraints required by modern payment gateways¹⁶.

Advanced Feature Engineering for Geospatial Risk Detection

Regardless of the underlying mathematical architecture, the efficacy of the AI Risk Manager is heavily dependent on the quality and creativity of its feature engineering. To accurately predict RTO and chargebacks, the model must ingest specialized signals, particularly concerning geospatial and address data.

In emerging markets like India, addresses are notoriously unstructured, highly variable, and lack strict standardization, making traditional geocoding highly ineffective. Advanced systems utilize Natural Language Processing (NLP) models, specifically Sequence-to-Sequence (Seq2Seq) geocoding approaches like the GeoIndia framework, to parse complex Indian addresses²⁵. These models evaluate address inputs for structural irregularities, such as "monkey-typed" gibberish, the absence of specific local landmarks, or mismatched pin code logic¹⁹. By predicting hierarchical spatial cells (e.g., H3-cell prediction), the system generates an "Address Quality Score," a vital feature that drastically improves the predictive power of the downstream RTO detection model¹⁹. Furthermore, velocity metrics (the number of orders or failed payment attempts within a specific time window) and device intelligence (detection of emulators, hidden VPNs, and persistent device fingerprint anomalies) form the foundational feature set required to separate legitimate users from sophisticated abusers¹¹.

Defensive AI: Differential Privacy and Evasion Robustness

The evaluation mandate for this specific hackathon strictly specifies a "defense-only" posture, dictating that any algorithmic mechanism capable of offensive operations (e.g., generating adversarial examples to exploit payment gateways, or models that can be reverse-engineered to extract synthetic identities) is grounds for immediate disqualification. This necessitates a rigid, provable adherence to privacy-preserving and robust machine learning principles.

The Threat of Offense-Capable Models

Standard machine learning models, particularly deep neural networks, have a well-documented propensity to inadvertently memorize their training data. In the context of fraud detection, the training dataset consists of highly sensitive Personally Identifiable Information (PII), financial records, device fingerprints, and behavioral logs. If an adversarial actor gains white-box access to the model's weights, or even black-box access to its prediction API, they could theoretically execute a *membership inference attack* to extract sensitive data. More dangerously, they could execute *model inversion* to reverse-engineer the exact feature thresholds and topological connections that trigger a fraud alert, subsequently using this knowledge to offensively generate synthetic "clean" identities that perfectly evade detection. A standard neural network, therefore, fails the "strictly defense-only" evaluation bar.

Differentially Private Stochastic Gradient Descent (DP-SGD)

To construct a provably defensive model, the AI Risk Manager must be trained using Differential Privacy (DP). Differential Privacy is a mathematically rigorous framework that provides a statistical guarantee that the inclusion or exclusion of any single training example (i.e., a single user's transaction history) does not significantly alter the output distribution of the model. This mathematically neutralizes the model's ability to memorize specific data points, thereby defeating membership inference and model inversion attacks⁷.

In practice, this is implemented via Differentially Private Stochastic Gradient Descent (DP-SGD), utilizing specialized open-source libraries such as PyTorch Opacus²⁸. DP-SGD modifies the standard deep learning training loop in two critical, mathematically sound ways:

1. **Per-Sample Gradient Clipping:** In standard SGD, gradients are averaged across a batch. In DP-SGD, the algorithm computes the gradient of the loss function with respect to each individual sample in the batch. It then clips the L_2 norm of these per-sample gradients to a predefined maximum threshold (e.g., `max_grad_norm`). This explicitly bounds the maximum influence any single transaction (or individual fraudster) can have on the model's learned parameters, preventing the model from over-indexing on an outlier⁸.
2. **Calibrated Noise Injection:** After the per-sample gradients are clipped and aggregated, random statistical noise (typically drawn from a Gaussian distribution, scaled by a `noise_multiplier` and the clipping threshold) is injected into the aggregate gradient before the model weights are updated⁸.

By injecting this calibrated noise, the model successfully learns the generalized, macro-level patterns of fraud (e.g., "rapid successive transactions from varying IPs indicate account takeover") without memorizing the specific credit card numbers, email addresses, or IP ranges associated with the training data. The privacy budget expended during the training process is

meticulously tracked using the mathematical parameters ϵ (epsilon, defining the privacy loss bound) and δ (delta, the probability that the bound is broken). Providing the final ϵ value alongside the model guarantees its defensive isolation, proving to evaluators that the system is incapable of offensive data generation⁸.

Standard Deep Learning (Offense-Capable)	Differentially Private Learning (Defense-Only)	Hackathon Evaluation Impact
Gradients averaged across the entire batch.	Per-sample gradients computed and explicitly clipped via L_2 norm.	Proves the model bounds the influence of any single actor.
Weights updated directly via gradient descent.	Gaussian noise injected into gradients before weight update.	Mathematically prevents the extraction of synthetic identities.
Vulnerable to Membership Inference Attacks.	Immune to Membership Inference Attacks (bounded by ϵ).	Satisfies the strict "defense-only" disqualification criteria.
Optimizes purely for maximum precision/recall.	Navigates the trade-off between privacy budget (ϵ) and accuracy.	Demonstrates real-world ML deployment maturity and discipline.

Robustness Against Adversarial Evasion

Fraud detection is inherently an adversarial domain. Bad actors actively probe defense mechanisms, attempting to find the optimal decision boundary to slip through undetected—a challenge formalized as the Adversarial Evasion problem³².

A strictly defensive AI Risk Manager counters evasion not by generating offensive adversarial examples to train against (which violates the hackathon constraint), but by implementing multi-layered structural defenses. By relying on a diverse ensemble—combining GNN structural topologies with Transformer behavioral sequences—the attacker is forced to simultaneously spoof their device footprint, their network origin, their sequential behavioral cadence, and their graph connections. Defeating one layer does not defeat the ensemble¹⁶. Furthermore, integrating unsupervised anomaly detection (e.g., Isolation Forests) alongside the supervised classifier establishes a baseline of normal behavior, ensuring that zero-day fraud tactics that have not yet resulted in labeled chargebacks are flagged purely based on their statistical

distance from the manifold of normal commerce³³.

Hackathon Execution Blueprint: Building and Presenting the AI Risk Manager

To successfully execute this complex problem statement and definitively surpass the evaluation criteria of the hackathon, the project must demonstrate extreme discipline in machine learning evaluation, cost-aware business logic, and defensive algorithmic integrity. The following blueprint outlines the exact execution strategy required to build the state-of-the-art product.

Phase 1: Data Curation, Synthesis, and Feature Engineering

Because proprietary financial data is highly confidential, the project requires a robust, publicly accessible proxy dataset. The IEEE-CIS Fraud Detection dataset, available via Kaggle, serves as an excellent foundational dataset for chargeback fraud, given its severe class imbalance (approximately 3.5% fraud) and incredibly rich tabular features (including DeviceType, TransactionAmt, and anonymized card details)⁹.

To demonstrate advanced RTO capabilities and satisfy the architectural requirements, this dataset must be augmented with synthetic graph structures and behavioral logs. This involves procedurally generating clusters of transactions that share identical device IDs, shipping addresses, or sequential timestamps to mathematically represent organized RTO rings and account takeover topologies. Furthermore, synthesize a text column representing Indian shipping addresses, allowing the demonstration of NLP-based address anomaly scoring¹⁹.

Phase 2: The Modeling Pipeline and Defensive Implementation

1. **Baseline Cost-Sensitive XGBoost:** Construct a baseline XGBoost classifier. Critically, implement the `scale_pos_weight` parameter to manage class imbalance natively, or utilize a custom focal loss function that penalizes the model heavily for missing the minority class³⁶. This proves baseline competence in handling imbalanced data.
2. **Advanced HGNN Architecture Implementation:** Utilizing libraries such as PyTorch Geometric, implement a lightweight version of the TRUST HGNN architecture. Construct nodes for users and transactions, and generate edges based on exact matches of device or IP metadata to create a multi-relational graph¹⁶.
3. **Defensive Guarantee (DP-SGD):** Wrap the PyTorch model optimizer with the Opacus

PrivacyEngine to enable DP-SGD. Log the privacy budget (ϵ) expended during each training epoch. This specific inclusion is the silver bullet that definitively proves the model is strictly defense-only and incapable of data memorization²⁹.

Phase 3: Honest Evaluation and Transparent Reporting

The defining element of the submission will not be the code itself, but the evaluation framework presented to the judges. The project must explicitly avoid the identified traps.

- **Discard Accuracy:** Begin the evaluation presentation by stating clearly that accuracy is a mathematically invalid metric for this domain.
- **Define the Cost Matrix:** Explicitly declare the business logic for the evaluation costs. For

example, establish that $C_{FP} = \$10$ (representing lost margin and lifetime value churn), and $C_{FN} = \$100$ (representing lost COGS, chargeback fees, and shipping losses)¹.

- **Implement BMR Thresholding:** Develop a Python module that computes Bayes Minimum Risk across the validation set. Generate a visualization that plots threshold values on the X-axis and total operational cost on the Y-axis. Demonstrate that shifting the decision threshold from the naive 0.5 to the empirically calculated optimal point (e.g., 0.12) minimizes total business loss¹.
- **Failure Mode Transparency:** Provide a detailed matrix of failure modes. Explicitly identify the types of transactions where the model struggles (e.g., "The model fails to detect highly sophisticated emulators that successfully spoof historical graph connections"). Calculate and explain the specific false-positive cost incurred when these misclassifications occur. This transparency fulfills the mandate for "no cherry-picked numbers."

Phase 4: The Auto-Responder Action Engine

Finally, connect the output of the machine learning risk score to a programmatic action engine to satisfy the "auto-responder" requirement. Create a tiered response framework based on the dynamically shifting BMR thresholds:

- **Score < τ_{low} (Low Risk):** Green-light. Proceed with frictionless payment processing.
- **Score > τ_{low} AND Score < τ_{high} (Medium Risk):** The Auto-Responder triggers Dynamic Friction. It holds the order and executes an API call to request verification (e.g., demands 3D-Secure authentication, SMS OTP, or nudges the user to a Part-Pay COD validation flow)¹⁷.
- **Score > τ_{high} (High Risk):** Hard block. The transaction is instantly declined to prevent a guaranteed chargeback or severe RTO logistics loss.

By executing this disciplined, cost-aware, and defensively rigid framework, the resulting project will not only solve the immediate technical challenge but will present a mature, production-ready AI Risk Manager that comfortably surpasses the most rigorous evaluation bars in the industry.

Works cited

1. Cost-Sensitive Threshold Optimization for Network Intrusion Detection: A Per-Class Approach with XGBoost - MDPI, <https://www.mdpi.com/2079-9292/15/7/1542>
2. A Cost Sensitive Learning Framework Using Xgboost For Financial Fraud Detection In Imbalanced Datasets - International Journal of Engineering and Techniques (IJET), <https://ijetjournal.org/cost-sensitive-learning-xgboost/>

3. How To Calculate The ROI of Your Decision Rules | Rippleshot,
<https://www.rippleshot.com/post/how-to-calculate-the-roi-of-your-decision-rules>
4. Fraud Detection Accuracy: What You Need to Know - Signifyd,
<https://www.signifyd.com/blog/fraud-detection-accuracy/>
5. How to Reduce RTO in E-commerce India: 7 Proven Strategies - 1Checkout,
<https://www.1checkout.ai/post/how-to-reduce-rto-in-e-commerce>
6. Example-dependent cost-sensitive decision trees - albahnsen,
<https://albahnsen.github.io/files/Example-Dependent%20Cost-Sensitive%20Decision%20Trees.pdf>
7. A Comprehensive Guide to Differential Privacy: From Theory to User Expectations - arXiv, <https://arxiv.org/html/2509.03294v3>
8. How do you apply differential privacy in machine learning? | by NS Academy - Medium,
<https://medium.com/@sharetonschool/how-do-you-apply-differential-privacy-in-machine-learning-56f8cfc91847>
9. Estimation of Optimal Threshold Shifting to Handle Class Imbalance in Credit Card Fraud Detection Using Machine Learning Techniques - AIP Publishing,
https://pubs.aip.org/aip/acp/article-pdf/doi/10.1063/5.0182386/18912460/120014_1_5.0182386.pdf
10. Handling Class Imbalance in Online Transaction Fraud Detection - Tech Science Press, <https://www.techscience.com/cmc/v70n2/44668/html>
11. Machine learning for fraud detection - Ravelin Technology,
<https://www.ravelin.com/insights/machine-learning-for-fraud-detection>
12. Predicting Fraud in Financial Payment Services through Optimized Hyper-Parameter-Tuned XGBoost Model - MDPI,
<https://www.mdpi.com/2227-7390/10/24/4679>
13. Foundations of Cost-Sensitive Learning | PDF | Statistical Classification - Scribd,
<https://www.scribd.com/document/483543209/20-Cost-Sensitive-Learning>
14. Cost-Sensitive Credit Card Fraud Detection Using Extreme Gradient Boosting and SMOTE on Imbalanced Financial Datasets - IRJEMS,
<https://irjems.org/Volume-5-Issue-4/IRJEMS-V5I4P122.pdf>
15. Tree-Based Cost Sensitive Methods for Fraud Detection in Imbalanced Data,
<https://www.semanticscholar.org/paper/Tree-Based-Cost-Sensitive-Methods-for-Fraud-in-Data-Metzler-Badiche/aa1ba3b58682d7ac68f6dab720e845fea91e348a>
16. TRUST: Transaction Risk via Unified Sequence and Topology,
<https://ojs.aaai.org/index.php/AAAI/article/view/41450/45411>
17. Blog - Dazeinfo, <https://dazeinfo.com/blog/page/7/>
18. 10 Best RTO Reduction Tools for D2C Brands in India (2026) - ClickPost,
<https://www.clickpost.ai/blog/rto-reduction-tools>
19. How to Reduce RTO in eCommerce: Causes & Solutions - Shiprocket Engage 360,
<https://www.shiprocket.in/blog/reduce-rto-in-ecommerce/>
20. Is 30% RTO, cost of doing e-commerce business? | by ThirdWatch | Medium,
<https://medium.com/@thirdwatch/is-30-rto-cost-of-doing-e-commerce-business-88dd4f89e31d>

21. eCommerce Fraud: Definition, Common Types, and Prevention Best, <https://www.chargeflow.io/chargebacks-101/ecommerce-fraud>
22. Predictive Analytics in Ecommerce: To Boost Sales & Optimise Conversions (1Checkout), <https://www.1checkout.ai/post/predictive-analytics-in-ecommerce>
23. Heterogeneous graph neural networks for fraud detection and explanation in supply chain finance | Request PDF - ResearchGate, https://www.researchgate.net/publication/378639126_Heterogeneous_graph_neural_networks_for_fraud_detection_and_explanation_in_supply_chain_finance
24. Benefits of Using AI in E-Commerce Fraud Detection - DxMinds, <https://dxminds.com/benefits-of-using-ai-in-ecommerce-fraud-detection/>
25. GeoIndia: Advanced Geocoding for India | PDF - Scribd, <https://www.scribd.com/document/854149647/2024-emnlp-industry-29>
26. Using Machine Learning to Detect Fraud: Introduction - Razorpay Tech, <https://razorpay.com/blog/detect-fraud-using-ml-ai-thirdwatch/>
27. Solving The RTO Challenge & Stopping Return Frauds - Bureau ID, <https://bureau.id/resources/blog/solving-the-rto-challenge>
28. Opacus: User-Friendly Differential Privacy Library in PyTorch - NeurIPS 2026, <https://neurips.cc/virtual/2021/32497>
29. GitHub - meta-pytorch/opacus: Training PyTorch models with differential privacy, <https://github.com/meta-pytorch/opacus>
30. Differentially Private Deep Learning in 20 lines of code: How to use the PyTorch Opacus Library - OpenMined, <https://openmined.org/blog/differentially-private-deep-learning-using-opacus-in-20-lines-of-code/>
31. Building a text classifier with Differential Privacy - Opacus, https://opacus.ai/tutorials/building_text_classifier
32. Classifier Evasion: Models and Open Problems - (EECS) at UC, <https://people.eecs.berkeley.edu/~adj/publications/paper-files/Nelson-Rubinstein-LNCS-2011.pdf>
33. Threat Control: Ransomware & Anomaly Detection for Panzura CloudFS, <https://panzura.com/products/threat-control>
34. Low-Rate DoS Attacks Detection Based on MAF-ADM - PMC, <https://pmc.ncbi.nlm.nih.gov/articles/PMC6982801/>
35. This repository implements Pozzolo, et al., (2015)'s probability calibration for imbalanced data. - GitHub, <https://github.com/kyosek/Probability-Calibration-Imbalanced>
36. GitHub - A-SHOJAEI/adaptive-threshold-calibration-fraud-detection, <https://github.com/A-SHOJAEI/adaptive-threshold-calibration-fraud-detection-with-cost-sensitive-curriculum>
37. A Mathematical Programming Model with Cost Sensitivity in the Objective Function for Imbalanced Datasets Challenges - Statistics, Optimization & Information Computing, <https://iapress.org/index.php/soic/article/download/2549/1370/10963>
38. heterogeneous-graph-neural-network · GitHub Topics, <https://github.com/topics/heterogeneous-graph-neural-network?o=asc&s=stars>

39. GitHub - 3015pavan/DRAGNN-FraudDB: An end-to-end Graph Neural Network (GNN) fraud detection pipeline leveraging PyTorch Geometric and PostgreSQL to identify complex financial crime patterns. Features a high-performance relational graph architecture and a real-time Streamlit analytics dashboard., <https://github.com/3015pavan/DRAGNN-FraudDB>
40. Opacus · Train PyTorch models with Differential Privacy, <https://opacus.ai/>